

Encontro 2: SQL Essencial

Criando e Consultando um Banco de Dados

PROJETO BIBLIOTECA

Estrutura

CREATE TABLE, tipos de dados e restrições

Manipulação

INSERT, SELECT, UPDATE e DELETE

Relacionamento

JOIN entre tabelas e integridade referencial

Agregação

COUNT, SUM, AVG e GROUP BY

Objetivos da Aula

1

CREATE TABLE

Criar tabelas com tipos de dados e restrições antes de qualquer inserção

2

INSERT / SELECT

Inserir, consultar, filtrar e ordenar dados com precisão

3

UPDATE / DELETE

Atualizar e excluir registros com segurança usando WHERE

4

JOIN

Relacionar tabelas com INNER JOIN e LEFT JOIN

5

Agregação

Agrupar e calcular com COUNT, SUM e AVG

Retomada: O Que É um Banco de Dados?

Conceitos Fundamentais

Banco de Dados

Coleção organizada de dados estruturados, armazenados e acessados eletronicamente

Tabela

Como uma planilha, mas com regras rígidas de formato - cada coluna tem tipo e restrições definidos

Linha

Um registro completo - ex.: um livro específico com todos os seus atributos

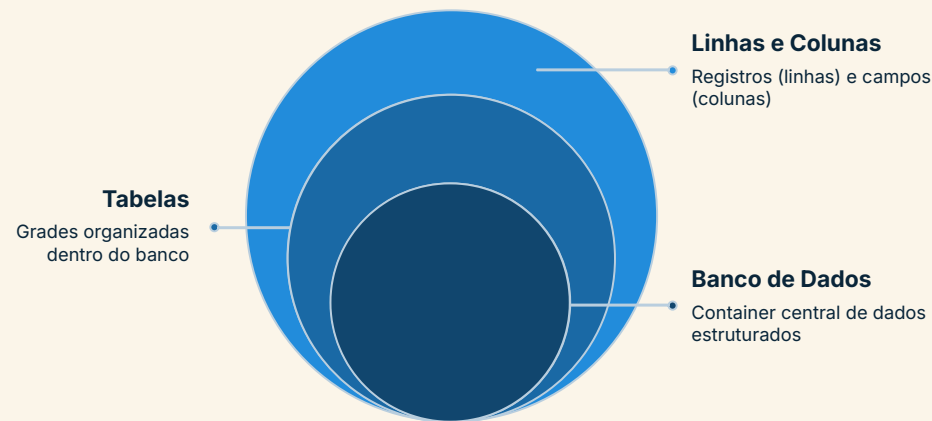
Coluna

Um campo com tipo definido - ex.: título (TEXT), preço (REAL), ano (INTEGER)



Pergunta disparadora: "Como o sistema da biblioteca sabe quais livros são de cada autor?"

A resposta está no modelo relacional: tabelas separadas conectadas por chaves. O campo **id_autor** na tabela de livros aponta para o registro correspondente na tabela de autores - sem duplicar nenhum dado.



Nosso Projeto: A Biblioteca - Modelagem de Dados

Para organizar os dados da nossa biblioteca, modelamos duas tabelas principais: **Autores** e **Livros**.

Tabela: Autores

Armazena informações sobre os escritores.

- **id_autor**: INTEGER (PK) - Identificador único do autor.
- **nome**: TEXT - Nome completo do autor.
- **nacionalidade**: TEXT - País de origem do autor.



1 para N

Tabela: Livros

Contém detalhes sobre cada obra literária.

- **id_livro**: INTEGER (PK) - Identificador único do livro.
- **titulo**: TEXT - Título do livro.
- **ano**: INTEGER - Ano de publicação.
- **preco**: REAL - Preço de venda.
- **categoria**: TEXT - Gênero literário (ex: Ficção, Poesia).
- **id_autor**: INTEGER (FK) - Liga o livro ao seu autor.

A relação **um para muitos** entre Autores e Livros significa que um autor pode ter vários livros, mas cada livro pertence a um único autor.

Nosso Projeto: A Biblioteca

O projeto usa duas tabelas relacionadas. O campo **id_autor** é a ponte entre elas - o coração do modelo relacional.

Tabela: autores

Coluna	Tipo	Papel
id_autor	INTEGER	PRIMARY KEY
nome	TEXT	NOT NULL
nacionalidade	TEXT	Opcional

Tabela: livros

Coluna	Tipo	Papel
id_livro	INTEGER	PRIMARY KEY
titulo	TEXT	NOT NULL
ano	INTEGER	-
preco	REAL	-
categoria	TEXT	-
id_autor	INTEGER	FOREIGN KEY - autores

❏ **FOREIGN KEY:** o campo **id_autor** em livros garante que todo livro pertença a um autor cadastrado. Dados sem repetição, com integridade garantida - esse vínculo é o coração do modelo relacional.

CREATE TABLE - Criando a Estrutura

O comando **CREATE TABLE** define colunas, tipos de dados e restrições antes de qualquer dado ser inserido.

Tabela autores

```
CREATE TABLE autores (  
  id_autor  INTEGER PRIMARY KEY,  
  nome      TEXT  NOT NULL,  
  nacionalidade TEXT  
);
```

Tabela livros

```
CREATE TABLE livros (  
  id_livro  INTEGER PRIMARY KEY,  
  titulo    TEXT  NOT NULL,  
  ano       INTEGER,  
  preco     REAL,  
  categoria TEXT,  
  id_autor  INTEGER,  
  FOREIGN KEY (id_autor)  
    REFERENCES autores(id_autor)  
);
```

1. Defina a estrutura primeiro

Nenhum dado pode ser inserido antes que a tabela exista com suas colunas e tipos definidos

2. Declare as restrições na criação

PRIMARY KEY, NOT NULL e FOREIGN KEY são declaradas junto com as colunas

3. Ordem importa

Crie **autores** antes de **livros** - a FOREIGN KEY exige que a tabela referenciada já exista

Tipos de Dados

O tipo de dado é a "regra de entrada" da coluna - tenta inserir texto em INTEGER e o banco rejeita. Escolher o tipo certo evita erros silenciosos como somar textos em vez de números.



INTEGER

Números inteiros sem casas decimais

- id, ano, quantidade
- Ex.: ano = 1997



TEXT

Cadeias de caracteres de qualquer tamanho

- nome, título, categoria
- Ex.: título = 'Dom Casmurro'



REAL

Números decimais com ponto flutuante

- preço, nota, média
- Ex.: preço = 49.90

⚠ Inserir texto em coluna INTEGER ou valor decimal em coluna INTEGER gera erro imediato - o banco rejeita o dado antes de gravá-lo.

Restrições - As Regras da Tabela

Restrições são a primeira linha de defesa contra dados inconsistentes. Elas são declaradas na criação da tabela e aplicadas automaticamente pelo banco a cada operação.

PRIMARY KEY

O "CPF" da linha - valor único e obrigatório que identifica cada registro. Não pode se repetir nem ser nulo.

FOREIGN KEY

Referência a outra tabela - impede cadastrar um livro com **id_autor** inexistente. Garante integridade referencial.

NOT NULL

Campo obrigatório - a coluna não pode ficar vazia. Ideal para campos essenciais como nome e título.

UNIQUE

Valor sem repetição na coluna - útil para e-mail, ISBN, CPF. Diferente de PRIMARY KEY: pode ser nulo.

INSERT - Adicionando Registros

O comando **INSERT INTO** popula a tabela linha a linha. A ordem dos valores deve seguir exatamente a ordem das colunas declaradas no CREATE TABLE.

Inserindo autores

```
INSERT INTO autores VALUES  
(1, 'J.K. Rowling', 'Britânica');  
INSERT INTO autores VALUES  
(2, 'Machado de Assis', 'Brasileiro');  
INSERT INTO autores VALUES  
(3, 'Hilda Hilst', 'Brasileira');
```

Inserindo livros

```
INSERT INTO livros VALUES  
(1, 'Harry Potter e a Pedra Filosofal',  
1997, 49.90, 'Fantasia', 1),  
(2, 'Dom Casmurro',  
1899, 35.00, 'Romance', 2),  
(3, 'Memórias Póstumas de Brás Cubas',  
1881, 32.00, 'Romance', 2);
```

Ordem dos valores

Deve seguir exatamente a ordem das colunas declaradas no CREATE TABLE

Violação de restrição

Inserir **id_autor** inexistente (ex.: 99) gera erro imediato - a FOREIGN KEY bloqueia



Atividade 1: Mão no Código

📋 **Missão:** criar a tabela **filmes** e inserir 5 registros.

🔵 Essencial

Modelo com lacunas:

```
CREATE TABLE filmes (  
  id __,  
  titulo __,  
  ano __,  
  nota __  
)
```

+ 5 INSERTs completos

🟡 Aprofundamento

Adicionar restrições à tabela criada:

- **NOT NULL** em título
- **UNIQUE** em título

Testar o que acontece ao inserir dois filmes com o mesmo título

🟢 Desafio

Turma: Tentem provocar intencionalmente um erro de restrição e entender a mensagem de erro

Ex.: inserir NULL em campo NOT NULL, duplicar PRIMARY KEY



10 min e compartilhamento

✓ Respostas: Atividade 1

Confira as soluções para a criação da tabela `filmes` e a inserção de registros, incluindo as restrições.

Criação da Tabela `filmes`

```
CREATE TABLE filmes (  
  id    INTEGER PRIMARY KEY,  
  titulo TEXT NOT NULL UNIQUE,  
  ano   INTEGER,  
  nota  REAL  
);
```

Inserção de Registros

```
INSERT INTO filmes VALUES  
(1, 'Matrix', 1999, 8.7),  
(2, 'O Poderoso Chefão', 1972, 9.2),  
(3, 'Interestelar', 2014, 8.6),  
(4, 'A Chegada', 2016, 7.9),  
(5, 'Clube da Luta', 1999, 8.8);
```

Restrições e Desafios

NOT NULL e UNIQUE em `titulo`

Garantem que o título não seja vazio e cada filme tenha um título exclusivo.

Violações de Restrições

Tentar inserir um filme com título já existente ou sem título irá gerar um erro SQL. O banco impede dados inválidos.

Provocando Erros

Insira `NULL` em um campo `NOT NULL`, ou duplique uma `PRIMARY KEY` ou `UNIQUE` para ver as mensagens de erro do sistema.

SELECT - Consultando Dados

O comando **SELECT** recupera dados da tabela. Pode trazer tudo ou apenas colunas específicas. **SELECT *** é prático para explorar, mas evite em produção - traz colunas desnecessárias.

Formas de uso

```
-- Tudo
SELECT * FROM livros;

-- Colunas específicas
SELECT titulo, preco
FROM livros;

-- Com alias
SELECT
    titulo AS "Título",
    preco AS "Preço (R$)"
FROM livros;
```

Boas práticas

SELECT * - exploração

Útil para inspecionar a tabela rapidamente durante o desenvolvimento

Colunas explícitas - produção

Nomear colunas torna o resultado mais legível, previsível e eficiente

Alias (AS)

Renomeia a coluna no resultado - não altera a estrutura da tabela

WHERE e Operadores - Filtrando Registros

Operadores disponíveis

Tipo	Operadores
Comparação	= < > <= >=
Lógicos	AND · OR · NOT
Lista	IN ('Fantasia', 'Romance')
Intervalo	BETWEEN 1800 AND 1900
Padrão	LIKE '%Harry%' - % é curinga

Exemplo combinado

```
SELECT titulo, preco
FROM livros
WHERE categoria = 'Fantasia'
AND preco > 40.00;
```

 O % no LIKE substitui qualquer sequência de caracteres. LIKE '%Harry%' encontra qualquer título que contenha "Harry" em qualquer posição.

Exemplos Práticos de Filtros

Veja como aplicar os operadores de filtro na prática para consultar os dados da nossa biblioteca.

1

COMPARAÇÃO

Filtrando por ano específico

```
SELECT titulo, ano
FROM livros
WHERE ano = 1997;
```

Retorna os títulos de livros publicados em 1997.

2

LÓGICOS

Combinando condições (OR)

```
SELECT titulo, categoria, ano
FROM livros
WHERE ano < 1900 OR categoria = 'Fantasia';
```

Busca livros anteriores a 1900 OU que sejam da categoria Fantasia.

3

LISTA

Selecionando de uma lista de categorias

```
SELECT titulo, categoria
FROM livros
WHERE categoria IN ('Romance', 'Fantasia');
```

Exibe livros cujas categorias são "Romance" ou "Fantasia".

4

INTERVALO

Filtrando por período

```
SELECT titulo, ano
FROM livros
WHERE ano BETWEEN 1990 AND 2000;
```

Encontra livros publicados entre 1990 e 2000, inclusive.

5

PADRÃO

Busca por parte do título

```
SELECT titulo
FROM livros
WHERE titulo LIKE '%Casmurro%';
```

Localiza livros que contêm a palavra "Casmurro" no título.

❏ Lembre-se que o `LIKE` com `%` é sensível ao case em alguns bancos de dados. Use `ILIKE` (PostgreSQL) para busca sem distinção de maiúsculas/minúsculas.

ORDER BY – Ordenando Resultados

O **ORDER BY** define a ordem de exibição das linhas retornadas. Múltiplas colunas: o segundo critério desempata o primeiro.

Exemplos de ordenação

-- Do mais antigo ao mais novo

```
SELECT titulo, ano  
FROM livros  
ORDER BY ano ASC;
```

-- Do mais caro ao mais barato

```
SELECT titulo, preco  
FROM livros  
ORDER BY preco DESC;
```

-- Múltiplos critérios

```
SELECT titulo, categoria, preco  
FROM livros  
ORDER BY categoria ASC, preco DESC;
```

Direções de ordenação

ASC

Crescente - padrão quando omitido. A → Z, 1 → 9, mais antigo → mais novo

DESC

Decrescente. Z → A, 9 → 1, mais novo → mais antigo

No exemplo com múltiplos critérios: livros são agrupados por categoria (A→Z) e, dentro de cada categoria, ordenados do mais caro ao mais barato.

Atividade 2: Consultas

 **Base:** tabelas autores e livros já populadas com os dados inseridos na Atividade 1.

Essencial

Listar todos os livros do mais antigo ao mais novo

Aprofundamento

Livros acima de R\$ 40 da categoria Fantasia, ordenados por preço decrescente

Desafio

Títulos que contenham "Harry" **OR** sejam anteriores a 1900, em ordem alfabética

 15 min

Atividade 2: Consultas

📄 **Base:** tabelas autores e livros já populadas com os dados inseridos na Atividade 1.

🔵 Essencial

Listar todos os livros do mais antigo ao mais novo

```
SELECT titulo, ano
FROM livros
ORDER BY ano ASC;
```

🟡 Aprofundamento

Livros acima de R\$ 40 da categoria Fantasia, ordenados por preço decrescente

```
SELECT titulo, preco
FROM livros
WHERE categoria = 'Fantasia'
AND preco > 40.00
ORDER BY preco DESC;
```

🟢 Desafio

Títulos que contenham "Harry" **OR** sejam anteriores a 1900, em ordem alfabética

```
SELECT titulo, ano
FROM livros
WHERE titulo LIKE '%Harry%'
OR ano < 1900
ORDER BY titulo ASC;
```

UPDATE e DELETE - Alterando e Removendo

⊗ ⚠ **Regra de segurança:** sempre use **WHERE**. Teste com **SELECT** antes de executar UPDATE ou DELETE.

Exemplos

-- Reajuste de 10% nos livros de Fantasia

UPDATE livros

SET preco = preco * 1.10

WHERE categoria = 'Fantasia';

-- Remover um livro específico

DELETE FROM livros

WHERE id_livro = 5;

Riscos sem WHERE

UPDATE sem WHERE

Altera *todos* os registros da tabela - sem exceção. Erro irreversível sem backup.

DELETE sem WHERE

Apaga *todos* os registros da tabela - a estrutura permanece, mas os dados somem.

Boas práticas

Use transações (**BEGIN / ROLLBACK / COMMIT**) em ambientes reais para poder desfazer erros.

Gerenciando Transações: BEGIN, ROLLBACK, COMMIT

Transações são cruciais para garantir a integridade dos dados. Elas permitem agrupar múltiplas operações em uma única unidade lógica: ou todas as operações são salvas no banco de dados (commit), ou nenhuma delas é (rollback), restaurando o estado anterior.

Fluxo da Transação

- **BEGIN;** Marca o início de uma transação. Todas as declarações SQL subsequentes são parte dela.
- **COMMIT;** Torna todas as alterações feitas dentro da transação permanentes no banco de dados.
- **ROLLBACK;** Desfaz todas as alterações feitas dentro da transação, revertendo o banco de dados ao estado antes do `BEGIN;`

❏ A prática de encapsular operações de modificação (INSERT, UPDATE, DELETE) em transações com **BEGIN** e **COMMIT/ROLLBACK** é fundamental em ambientes de produção para evitar perda de dados e garantir a consistência.

Inserindo mais Livros

Para enriquecer nossas consultas com dados variados, vamos adicionar vários livros clássicos à nossa tabela livros, cadastrando também os respectivos autores.

-- Primeiro, cadastramos os novos autores

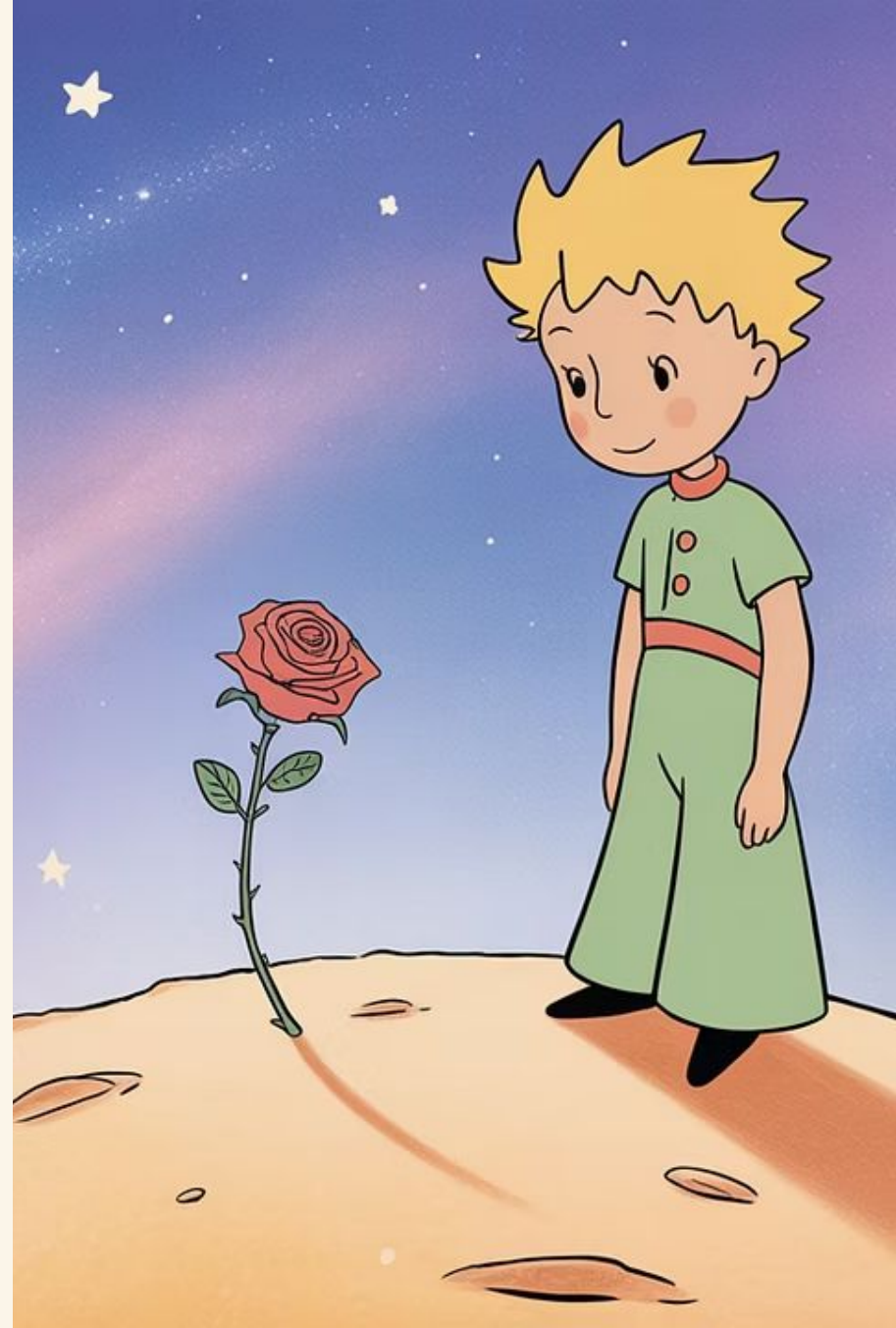
`INSERT INTO` autores `VALUES` (4, 'Antoine de Saint-Exupéry', 'Francês');

`INSERT INTO` autores `VALUES` (5, 'Franz Kafka', 'Tcheco');

`INSERT INTO` autores `VALUES` (6, 'Fiódor Dostoiévski', 'Russo');

`INSERT INTO` autores `VALUES` (7, 'Gabriel García Márquez', 'Colombiano');

A seguir, inserimos os livros vinculados a cada autor.



Inserindo os Livros

Com os autores cadastrados, agora inserimos os livros respeitando a ordem das colunas: id_livro → titulo → ano → preco → categoria → id_autor.

```
-- (id_livro, titulo, ano, preco, categoria, id_autor)
INSERT INTO livros VALUES (4, 'O Pequeno Príncipe',      1943, 35.00, 'Literatura Infantil', 4);
INSERT INTO livros VALUES (5, 'A Metamorfose',          1915, 29.90, 'Romance',          5);
INSERT INTO livros VALUES (6, 'Crime e Castigo',         1866, 42.00, 'Romance',          6);
INSERT INTO livros VALUES (7, 'Cem Anos de Solidão',     1967, 48.50, 'Realismo Mágico',   7);
INSERT INTO livros VALUES (8, 'O Processo',              1925, 31.00, 'Romance',          5);
INSERT INTO livros VALUES (9, 'O Idiota',                1869, 39.90, 'Romance',          6);
```

❌ ⚠️ O campo id_autor deve referenciar um autor já existente na tabela autores. Inserir um id_autor inexistente causará um erro de integridade referencial (FOREIGN KEY).

Transações na Prática: Exemplos com COMMIT e ROLLBACK

Veja como encapsular operações de UPDATE e DELETE em transações para testar com segurança antes de confirmar.

UPDATE Seguro com COMMIT

```
BEGIN;  
  
-- Simula uma atualização de preço para 'O Pequeno Príncipe'  
UPDATE livros  
SET preco = 65.00  
WHERE titulo = 'O Pequeno Príncipe';  
  
-- Confirma a alteração, tornando-a permanente  
COMMIT;  
  
-- O preço do livro 'O Pequeno Príncipe' é agora R$ 65.00 no  
banco de dados.
```

Use COMMIT quando tiver certeza de que as alterações devem ser salvas de forma definitiva.

DELETE Teste com ROLLBACK

```
BEGIN;  
  
-- Simula a remoção de todos os livros da categoria 'Infantil'  
DELETE FROM livros  
WHERE categoria = 'Literatura Infantil';  
  
-- Verifique os dados antes de confirmar:  
-- SELECT * FROM livros WHERE categoria = 'Literatura Infantil';  
  
-- Desfaz a remoção, restaurando os livros  
ROLLBACK;  
  
-- Todos os livros da categoria 'Infantil' são restaurados.
```

Use ROLLBACK para desfazer operações, seja por um erro ou por um teste que não deve ser salvo.

INNER JOIN e LEFT JOIN — Relacionando Tabelas

O JOIN combina linhas de duas tabelas com base em uma coluna em comum. No nosso projeto, a coluna `id_autor` é a ponte entre `livros` e `autores`.

INNER JOIN

Retorna apenas os livros que possuem um autor cadastrado. Se um livro tiver `id_autor` inexistente na tabela `autores`, ele não aparece no resultado.

```
-- Passo 1: o banco lê a tabela livros
-- Passo 2: para cada linha, busca o autor em autores
--      onde livros.id_autor = autores.id_autor
-- Passo 3: retorna apenas as linhas com correspondência
```

```
SELECT livros.titulo, autores.nome
FROM livros
INNER JOIN autores
ON livros.id_autor = autores.id_autor;
```

Resultado: 9 linhas — Hilda Hilst não aparece pois nenhum livro foi vinculado a ela.

LEFT JOIN

Retorna todos os autores, mesmo os que ainda não têm livros cadastrados. Onde não há correspondência, o banco preenche com `NULL`.

```
-- Passo 1: o banco lê a tabela autores (lado esquerdo)
-- Passo 2: para cada autor, tenta encontrar livros
--      onde autores.id_autor = livros.id_autor
-- Passo 3: se não encontrar, preenche titulo com NULL
```

```
SELECT autores.nome, livros.titulo
FROM autores
LEFT JOIN livros
ON autores.id_autor = livros.id_autor;
```

Resultado: Hilda Hilst aparece com `titulo = NULL` — ela está cadastrada, mas sem livros vinculados ainda.

  Momento 'aha': a diferença entre INNER e LEFT JOIN fica clara com Hilda Hilst. No INNER JOIN ela some; no LEFT JOIN ela aparece com `NULL`. Isso mostra o poder do modelo relacional: os dados existem independentemente, e o JOIN decide como combiná-los.

JOIN na Prática — Combinando com WHERE e ORDER BY

Agora que entendemos como o JOIN conecta as tabelas, podemos combiná-lo com cláusulas que já conhecemos para responder perguntas mais específicas sobre o acervo.

Filtrando após o JOIN

Podemos usar `WHERE` depois do JOIN para filtrar o resultado combinado. O banco primeiro une as tabelas, depois aplica o filtro.

```
-- Quais livros de Romance têm autor cadastrado?  
-- Passo 1: JOIN une livros e autores  
-- Passo 2: WHERE filtra apenas a categoria 'Romance'
```

```
SELECT livros.titulo, autores.nome, livros.ano  
FROM livros  
INNER JOIN autores  
  ON livros.id_autor = autores.id_autor  
WHERE livros.categoria = 'Romance';
```

Resultado: Dom Casmurro, Memórias Póstumas, Crime e Castigo, O Processo, O Idiota — todos com seus respectivos autores.

Ordenando o resultado do JOIN

O `ORDER BY` funciona normalmente após o JOIN — basta indicar por qual coluna ordenar.

```
-- Listar todos os livros com autor, do mais antigo ao mais novo  
-- Passo 1: JOIN une as tabelas  
-- Passo 2: ORDER BY ordena o resultado final
```

```
SELECT livros.titulo, autores.nome, livros.ano  
FROM livros  
INNER JOIN autores  
  ON livros.id_autor = autores.id_autor  
ORDER BY livros.ano ASC;
```

Resultado: A Metamorfose (1915) aparece antes de Dom Casmurro (1899)? Não — `ORDER BY` respeita o valor numérico: 1866 → 1869 → 1881 → 1899 → 1915 → ...

  **A ordem das cláusulas importa:** `FROM` → `JOIN` → `WHERE` → `ORDER BY`. O banco executa nessa sequência — primeiro une, depois filtra, depois ordena. Nunca o contrário.

COUNT, SUM, AVG + GROUP BY

Funções de agregação calculam sobre grupos de linhas. O **GROUP BY** divide o resultado em grupos antes de agregar - cada categoria vira uma linha de resumo.

Query de agregação

```
SELECT
  categoria,
  COUNT(*) AS total_livros,
  AVG(preco) AS preco_medio,
  SUM(preco) AS valor_total
FROM livros
GROUP BY categoria
ORDER BY total_livros DESC;
```

Funções disponíveis

COUNT(*)

Conta o número de linhas do grupo - inclui todas as linhas, mesmo com NULL

SUM(preco)

Soma os valores numéricos do grupo - valor total do acervo por categoria

AVG(preco)

Média aritmética dos valores do grupo - preço médio por categoria

GROUP BY

Agrupar linhas com o mesmo valor na coluna especificada antes de aplicar as funções

Refinando Agregações: ROUND e COUNT(DISTINCT)

Melhore a precisão e a utilidade dos seus resultados de agregação com estas dicas essenciais.

Arredondando Valores com ROUND()

A função `ROUND()` permite especificar o número de casas decimais para valores numéricos, ideal para apresentar preços e médias de forma mais legível.

```
SELECT
  categoria,
  COUNT(*) AS total_livros,
  ROUND(AVG(preco), 2) AS preco_medio_arredondado,
  SUM(preco) AS valor_total
FROM livros
GROUP BY categoria
ORDER BY total_livros DESC;
```

O `ROUND(AVG(preco), 2)` arredonda a média do preço para duas casas decimais, garantindo que o resultado seja formatado como moeda.

Contando Valores Únicos com COUNT(DISTINCT)

Para saber quantos valores únicos existem em uma coluna dentro de um grupo (ex: quantos autores diferentes publicaram em uma categoria), use `COUNT(DISTINCT coluna)`.

```
SELECT
  categoria,
  COUNT(DISTINCT id_autor) AS autores_unicos,
  COUNT(*) AS total_livros
FROM livros
GROUP BY categoria
ORDER BY autores_unicos DESC;
```

Esta query retorna o número de autores distintos por categoria, útil para analisar a diversidade de colaboradores.

Desafio Integrador

 **Missão:** "Quanto vale o acervo de cada autor e quem está com o acervo vazio?"

Essencial

Contar o total de livros por categoria

Aprofundamento

Calcular a média de preço por categoria

Desafio

Nome do autor + quantidade de livros +
valor total, incluindo autores sem livro

 **30 min** - resultado esperado: Hilda Hilst aparece com **0 livros** e **R\$ 0,00**



Desafio Integrador

📋 **Missão:** "Quanto vale o acervo de cada autor e quem está com o acervo vazio?"

🔵 Essencial

Contar o total de livros por categoria

```
SELECT categoria,  
       COUNT(*) AS total  
FROM livros  
GROUP BY categoria;
```

🟡 Aprofundamento

Calcular a média de preço por categoria

```
SELECT categoria,  
       AVG(preco) AS preco_medio  
FROM livros  
GROUP BY categoria;
```

🟢 Desafio

Nome do autor + quantidade de livros + valor total, incluindo autores sem livro

```
SELECT a.nome,  
       COUNT(l.id_livro) AS qtd_livros,  
       COALESCE(SUM(l.preco), 0) AS  
       valor_total  
FROM autores a  
LEFT JOIN livros l  
  ON a.id_autor = l.id_autor  
GROUP BY a.nome;
```

Resultado esperado: Hilda Hilst aparece com **0 livros** e **R\$ 0,00**

Encerramento e Ticket de Saída

Síntese - Ordem das Cláusulas

1

SELECT

Quais colunas retornar

2

FROM

De qual tabela

3

WHERE

Filtrar linhas

4

GROUP BY

Agrupar para agregar

5

ORDER BY

Ordenar o resultado

Ticket de Saída *(individual, 3 min)*

01

CREATE TABLE

Escrever o comando para criar uma tabela simples com pelo menos 3 colunas e uma PRIMARY KEY

02

SELECT com WHERE

Escrever um SELECT que filtre registros usando pelo menos um operador de comparação

03

INNER JOIN vs LEFT JOIN

Explicar a diferença entre os dois tipos de JOIN em uma única frase



Próximo encontro: subconsultas, índices e modelagem de um banco mais complexo